

IT3020 – Database System Laboratory

Practical 08 – Transaction and Concurrency Control Simulations

Student Name: Sewmini A A M Y

Student ID: IT23602946

Batch/Group: 3.2_WE

Date: 2026/04/10

1. Introduction

The objective of this laboratory exercise is to demonstrate the fundamental properties of database transactions, specifically Atomicity, and to understand Concurrency Control and Deadlock scenarios using MS SQL Server.

2. Environment Setup

To begin the simulation, a bank database was created with an account table containing two specific accounts.

-- Create Database

CREATE DATABASE Bank;

GO

USE Bank;

-- Create Account Table

```
CREATE TABLE Account (  
    AccountNo VARCHAR(10) PRIMARY KEY,  
    Balance DECIMAL(10, 2)  
);
```

-- Adding Constraint: Balance must be ≥ 0

```
ALTER TABLE Account ADD CONSTRAINT CHK_Balance CHECK (Balance  $\geq$  0);
```

-- Initial Data Insertion

```
INSERT INTO Account VALUES ('1234567899', 100.00);
```

```
INSERT INTO Account VALUES ('1234567898', 100.00);
```

--Table inserted correctly

```
SELECT * FROM Account;
```

Results		Messages	
	AccountNo	Balance	
1	1234567898	100.00	
2	1234567899	100.00	

03. Simulation 1: Atomicity

Objective: To verify that a transaction is an all-or-nothing operation.

```
SET XACT_ABORT ON; -- Ensures atomicity
```

```
BEGIN TRANSACTION;
```

```
DECLARE @sourceAcc VARCHAR(10) = '1234567899';
```

```
DECLARE @destAcc VARCHAR(10) = '1234567898';
```

```
DECLARE @fundsToTransfer DECIMAL(10, 2) = 150.00;
```

```
-- Credit destination account first
```

```
UPDATE Account SET Balance = Balance + @fundsToTransfer WHERE AccountNo = @destAcc;
```

```
-- Deduct from source account
```

```
UPDATE Account SET Balance = Balance - @fundsToTransfer WHERE AccountNo = @sourceAcc;
```

```
COMMIT TRANSACTION;
```

```
Msg 547, Level 16, State 0, Line 34
The UPDATE statement conflicted with the CHECK constraint "CHK_Balance". The conflict occurred in database "Bank", table "dbo.Account", column 'Balance'.
Completion time: 2026-04-09T22:48:31.9682646+05:30
```

Observation and Explanation:

- **Result:** The transaction fails with a "CHECK constraint conflict" error.
- **Reason:** The source account only has Rs. 100. Attempting to deduct Rs. 150 violates the CHK_Balance constraint.
- **Atomicity:** Due to SET XACT_ABORT ON, the entire transaction is rolled back. Even though the first UPDATE succeeded temporarily, the failure of the second UPDATE caused the whole process to abort, leaving account balances unchanged at Rs. 100 each.

SELECT * FROM Account;

Results Messages		
	AccountNo	Balance
1	1234567898	100.00
2	1234567899	100.00

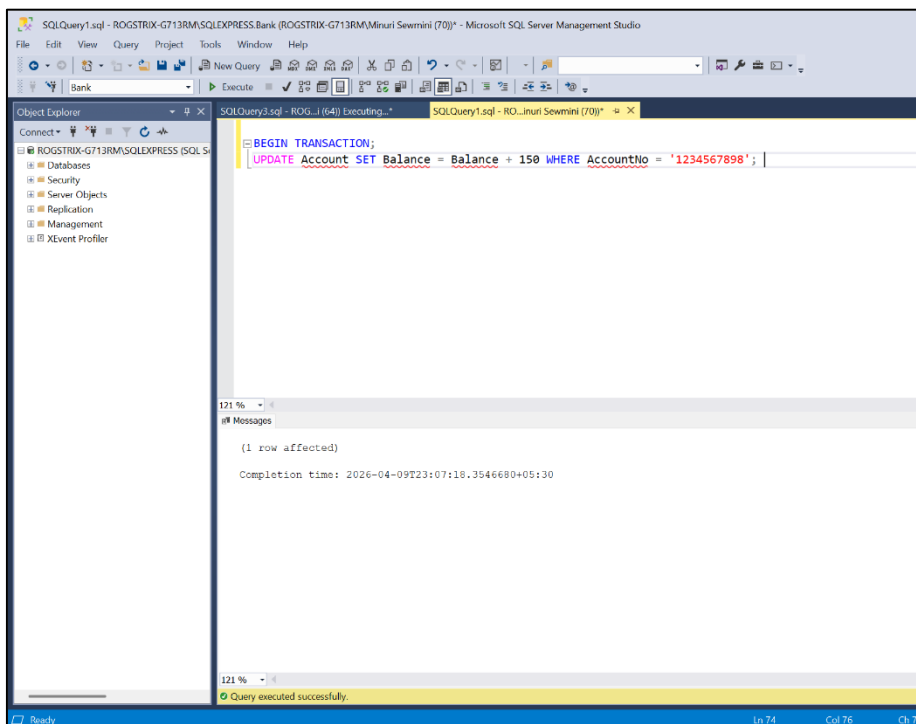
4. Simulation 2: Concurrency Control

Objective: To observe how SQL Server manages simultaneous access to the same data using locks.

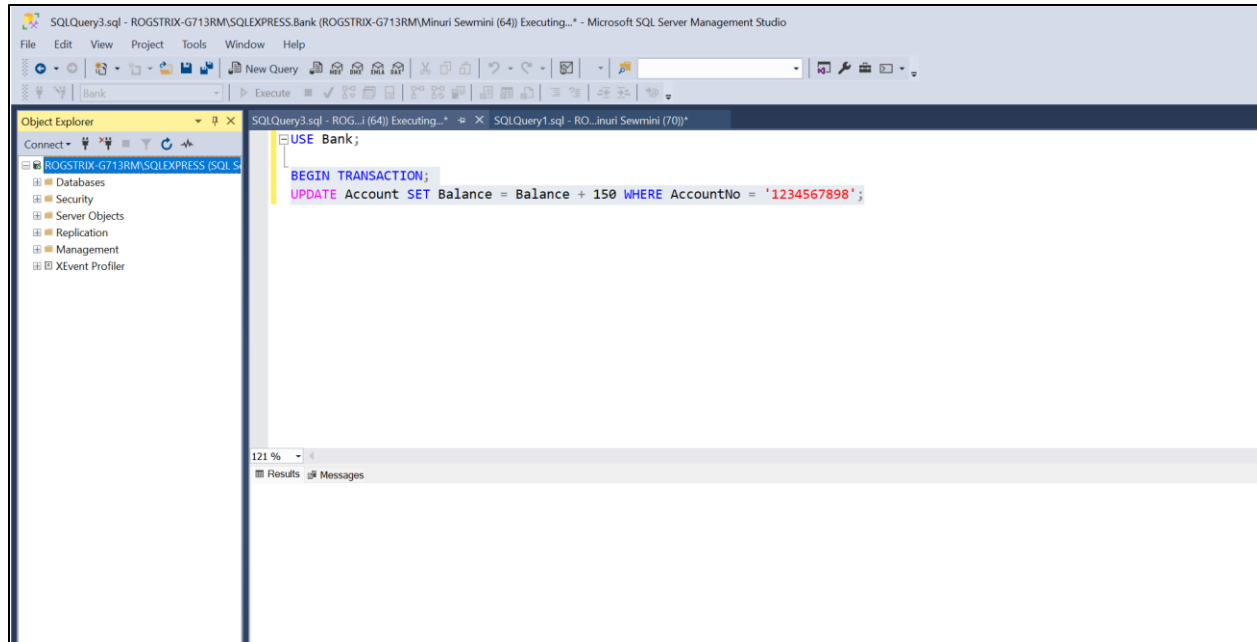
Procedure:

1. **Connection A:** Execute BEGIN TRANSACTION and the first UPDATE.
2. **Connection B:** Attempt to execute the same UPDATE on the same record.

Connection A



Connection B



Observation:

- **Behavior:** Connection B hangs and remains in "Executing" mode.
- **Reason:** Connection A holds an Exclusive Lock (X-Lock) on the row. Connection B must wait for Connection A to either COMMIT or ROLLBACK before it can access the data.

5. Simulation 3: Deadlock

Objective: To simulate a scenario where two transactions are waiting for each other, causing a circular dependency.

Setup:

Using SERIALIZABLE isolation level for both connections.

Steps:

- **Transaction 1 (Window A):** Updates Account '99'.
- **Transaction 2 (Window B):** Updates Account '98'.

- **Conflict:** Transaction 1 tries to update Account '98' (locked by B), and Transaction 2 tries to update Account '99' (locked by A).

Procedure:

1. **Transaction A:** Updates Account '99'.

UPDATE Account SET Balance = Balance + 10 WHERE AccountNo = '1234567899';

2. **Transaction B:** Updates Account '98'.

UPDATE Account SET Balance = Balance + 10 WHERE AccountNo = '1234567898';

3. **Transaction A:** Attempts to update Account '98' (Blocked by B).

UPDATE Account SET Balance = Balance + 10 WHERE AccountNo = '1234567898';

4. **Transaction B:** Attempts to update Account '99' (Blocked by A).

UPDATE Account SET Balance = Balance + 10 WHERE AccountNo = '1234567899';

Result:

- **Error:** SQL Server detects the deadlock and terminates one transaction as a **Deadlock Victim**.
- **Message:** "Msg 1205... Transaction was deadlocked on lock resources with another process and has been chosen as the deadlock victim."

Msg 1205, Level 13, State 51, Line 4
Transaction (Process ID 64) was deadlocked on lock resources with another process and has been chosen as the deadlock victim. Rerun the transaction.

Completion time: 2026-04-09T23:30:45.8397955+05:30

06.Conclusion

These simulations successfully demonstrated that MS SQL Server maintains data integrity through Atomicity and manages concurrent access using robust locking mechanisms and deadlock detection.